1. Introduction to Python

Theory:-

1. Introduction to Python and its Features

Python is a simple, high-level, and interpreted programming language. It is easy to learn and widely used for web development, data science, artificial intelligence, and automation.

➢ Features of Python

- Simple and easy syntax
- High-level language
- Interpreted language
- Portable and open-source
- Supports object-oriented programming

2. History and Evolution of Python

Python was created by **Guido van Rossum** in 1991. It was developed to provide a simple and readable programming language. Over time, Python became popular because of its simplicity and powerful libraries.

3. Advantages of Python over Other Languages

- Easy to learn and use
- Less coding compared to other languages
- Large collection of libraries
- Platform independent
- Useful for AI, machine learning, and web development

4. Installing Python and Setting Up Environment

1. Download Python from the official website.
2. Install Python and enable “Add Python to PATH”.
3. Install an IDE such as PyCharm, VS Code, or Anaconda.
4. Configure the IDE for Python programming.

5. Writing and Executing First Python Program

```
print("hello \"tops")

print('hello " tops')
print("hello ' tops")
print("he\tll\bo \" tops")
```

- **Steps to Run**

1. Open IDE or terminal
2. Write the program
3. Save file with `.py` extension
4. Run the program
5. Output will display:

```
hello "tops

hello " tops

hello ' tops

he      lo " tops
```

Lab Exercise

1. Write a Python program that prints "Hello, World!".

Program

```
print("\"Hello, World!\"")
```

Output

```
"Hello, World!"
```

2. Set up Python on your local machine and write a program to display your name.

Program

```
print("My name is Vivek kaklotar")
```

Output

```
My name is Vivek kaklotar
```

2. Programming Style

Theory:-

1. What are Python's PEP 8 Guidelines?

PEP 8 is the official style guide for Python programming. It helps programmers write clean, readable, and consistent code.

2. Explain indentation, comments, and naming conventions in Python.

- **Indentation** is used to define blocks of code in Python.
- **Comments** are used to explain the code and improve understanding.
- **Naming conventions** suggest meaningful variable and function names using lowercase letters and underscores.

Example:

```
student_name = "Vivek"
```

3. Why is readable and maintainable code important?

Readable and maintainable code is easy to understand, debug, and modify. It improves teamwork and reduces programming errors.

Lab Exercise

Write a Python program that demonstrates proper indentation, comments, and variables following PEP 8 guidelines.

Program

```
c='y'

while(c=='y'):
```

```
a = int (input("enter number 1:"))
b = int (input("enter number 2:"))

choice = input ("enter choice: +, -, *, /")
match choice:
    case "+":
        print(f"addition of {a} and {b} is {a+b}")
    case "-":
        print(f"subtraction of {a} and {b} is {a-b}")
    case "*":
        print(f"multiplication of {a} and {b} is {a*b}")
    case "/":
        print(f"division of {a} and {b} is {a/b}")
    case _:
        print("Invalid choice")

c = input("do you want to continue: y/n")
```

Output

```
enter number 1:20

enter number 2:30

enter choice: +, -, *, /+

addition of 20 and 30 is 50

do you want to continue: y/n
```

3. Core Python Concepts

Theory:-

1. Explain Python Data Types.

Python supports different data types:

- **Integer (int)** → stores whole numbers
- **Float (float)** → stores decimal numbers
- **String (str)** → stores text
- **List** → ordered and changeable collection
- **Tuple** → ordered but unchangeable collection
- **Dictionary** → stores data in key-value pairs
- **Set** → unordered collection of unique values

Example:

```
x = 10
name = "Vivek"
```

2. Explain Python Variables and Memory Allocation.

Variables are used to store data values in Python. Python automatically allocates memory when a variable is created.

Example:

```
age = 20
```

3. Explain Python Operators.

Python operators are used to perform operations on values and variables.

- **Arithmetic Operators** → `+ , - , * , /`
- **Comparison Operators** → `== , != , > , <`
- **Logical Operators** → `and , or , not`
- **Bitwise Operators** → `& , | , ^`

Lab Exercise:-

1. Write a Python program to demonstrate variables and data types.

Program

```
# Variables and data types

#numeric data types

id = 10
print(type(id))                                     #int (immutable)
print(id)

price = 99.99                                       #float (immutable)
print(type(price))
print(price)

a = 10 + 5j                                         #complex number (immutable)
print(type(a))
print(a)
#sequence data types

first_name = "vivek"                                #string (immutable)
print(type(first_name))
print(first_name)

subjects = ["python", "java", "c++"]                #list (mutable)
print(type(subjects))
print(subjects)

subjects = ("python", "java", "c++")                #tuple (immutable)
print(type(subjects))
print(subjects)

subjects = {"python", "java",
"c++","python"}                                     #set(remove duplicate values)
print(type(subjects))
print(subjects)

subjects = {"python": 90, "java": 85, "c++":
80}                                                 #dictionary (key-value pair)
```

```
print(type(subjects))
print(subjects)

#map data types
person = {"namae": "vivek", "email":
"vivek@gmail.com"}                                  #dictionary (key-value pair)
print(type(person))
print(person)

k = {1,2,2,3,4,5,6,7,7,8}                            #set (remove duplicate
values)
print(type(k))
print(k)

#boolean data types
is_valid = True                                      #boolean (True or False)
print(type(is_valid))
print(is_valid)

x = None                    #NoneType (used to represent the absence of a value)
print(type(x))
print(x)
```

Output:-

```
<class 'int'>
10
<class 'float'>
99.99
<class 'complex'>
(10+5j)
<class 'str'>
vivek
<class 'list'>
['python', 'java', 'c++']
<class 'tuple'>
('python', 'java', 'c++')
<class 'set'>
{'python', 'java', 'c++'}
<class 'dict'>
{'python': 90, 'java': 85, 'c++': 80}
<class 'dict'>
{'namae': 'vivek', 'email': 'vivek@gmail.com'}
<class 'set'>
{1, 2, 3, 4, 5, 6, 7, 8}
<class 'bool'>
True
<class 'NoneType'>
None
```

2. Practical Example 1: How does Python code structure work?

Program

```
# Python structure

print("vivek kaklotar")
age = 22
if 18 <= age <= 60:
    print("age is eligible")
else:
    print("not eligible")
```

Output

```
vivek kaklotar
age is eligible
```

3. Practical Example 2: How to create variables in Python?

Program

```
my_name = "Vivek"
my_marks = 85

print(my_name)
print(my_marks)
```

Output

```
Vivek
85
```

4. Practical Example 3: How to take user input using the `input()` function?

Program

```
name = input("Enter name: ")

print(name)
```

Output

```
Enter name: Vivek
Vivek
```

5. Practical Example 4: How to check the type of a variable dynamically using `type()`?

Program

```
age = 22

print(type(age))
```

Output

```
<class 'int'>
```

4. Conditional Statements

Theory

1. What are conditional statements in Python?

Conditional statements are used to make decisions in a program based on conditions.

- `if` → executes code if condition is true
- `else` → executes code if condition is false
- `elif` → checks multiple conditions

Example:

```
age = 18

if age >= 18:
    print("Eligible")
else:
    print("Not Eligible")
```

2. What are nested if-else conditions?

Nested if-else means using an `if-else` statement inside another `if-else` statement.

Example:

```
age = 20

if age >= 18:
    if age >= 21:
        print("Adult")
```

Lab Exercise:-

1. Practical Example 5: Write a Python program to find greater and less than a number using if-else.

Program

```
num = int(input("Enter a number: "))

if num > 10:
    print("Number is greater")
else:
    print("Number is less than")
```

Output

```
Enter a number: 11

Number is greater
```

2. Practical Example 6: Write a Python program to check if a number is prime using if-else.

Program

```
number = 17
flag = 0
for i in range (2,number):
    if number%i==0:
        flag=1
        break
if flag==0:
    print(f"{number} is a prime")
else:
    print(f"{number} is not a prime")
```

Output

```
17 is a prime
```

3. Practical Example 7: Write a Python program to calculate grades based on percentage using if-else ladder.

Program

```
c = 'y'
while (c=='y'):
    marks = int (input("enter your marks:  "))
    if marks >= 91 and marks <= 100:
        print("Grade A (PASS)")
    elif marks >= 71 and marks <= 90:
        print("Grade B (PASS)")
    elif marks >= 51 and marks <= 70:
        print("Grade C (PASS)")
    elif marks >= 35 and marks <= 50:
        print("Grade D (PASS)")
    elif marks >= 0 and marks <= 34:
        print("Grade F (FAIL)")
    else:
        print("Not valid marks")
    c = input("Do you want to continue: y/n : ")
```

Output

```
enter your marks:  68

Grade C (PASS)

Do you want to continue: y/n :
```

4. Practical Example 8: Write a Python program to check if a person is eligible to donate blood using nested if.

Program

```
age = int(input("Enter age: "))

if age >= 18:
        print("Eligible")
else:
    print("Not eligible")
```

Output

```
Enter age: 22
Eligible
```

5. Looping (For, While)

Theory:-

1. What are for and while loops in Python?

- **for loop** is used to repeat code for a fixed sequence or collection.
- **while loop** is used to repeat code while a condition is true.

2. How do loops work in Python?

Loops execute a block of code repeatedly until the condition becomes false or the sequence ends.

3. How are loops used with collections?

Loops can iterate through collections like lists, tuples, and strings.

Lab Exercise:-

1. Practical Example 1: Write a Python program to print each fruit in a list using a simple for loop.

Program

```
list = ["apple", "banana", "mango"]

for all_fruit in list:
    print(all_fruit)
```

Output

```
apple
banana
mango
```

2. Practical Example 2: Write a Python program to find the length of each string in List1.

Program

```
list = ["apple", "banana", "mango"]

for all_fruit in list:
    print(all_fruit,len(all_fruit))
```

Output

```
apple 5
banana 6
mango 5
```

Practical Example 3: Write a Python program to find a specific string in the list using for loop and if condition.

Program

```
list1 = ["vivek","yash"]

for name in list1:
    if name == "vivek":
        print("right name")
```

Output

```
right name
```

Practical Example 4: Print the pattern using nested for loop.

Program

6. Generation and Iterators

Theory:-

1. What are generators in Python?

Generators are functions that generate values one at a time using the `yield` keyword.

2. Difference between yield and return.

Yield	return
Returns values one by one	Returns value once
Used in generators	Used in normal functions
Saves memory	Ends function execution

3. What are iterators?

Iterators are objects used to iterate through elements one by one.

Lab Exercise:-

1. Write a generator function to generate the first 10 even numbers.

Program

Write a Python program that uses a custom iterator to iterate over a list.

Program

7. Functions and Methods

Theory:-

1. What are functions in Python?

Functions are blocks of code used to perform specific tasks.

2. What are function arguments?

Arguments are values passed to functions.

Types:

- Positional
- Keyword
- Default

3. What is variable scope in Python?

Scope defines where a variable can be accessed.

- Local scope
- Global scope

4. What are built-in methods?

Built-in methods are predefined functions for strings, lists, etc.

Lab Exercise:-

1. Practical Example 1: Print "Hello" using a string.

Program

```
name = "hello"
print(name)
```

Output

```
hello
```

2. Practical Example 2: Locate a string to a variable and print it.

Program

```
name = "vivek"
print(name)
```

Output

```
vivek
```

3. Practical Example 3: Print a string using triple quotes.

Program

```
name = """hello vivek"""
print(name)
```

Output

```
hello vivek
```

4. Practical Example 4: Access the first character of a string using index value.

Program

```
name = "Vivek"
print(name[0])
```

Output

```
V
```

5. Practical Example 5: Access the fifth character of a string.

Program

```
name = "Vivek"
print(name[4])
```

Output

```
k
```

6. Practical Example 6: Print the substring between index 1 and 4.

Program

7. Practical Example 7: Print a string from the last character.

Program

```
name = "Vivek"
print(name[-1])
```

Output

```
k
```

8. Practical Example 8: Print every alternate character from the string.

Program

Output

8. Control Statements (Break, Continue, Pass)

Theory:-

1. What are break, continue, and pass statements?

- `break` → stops the loop
- `continue` → skips current iteration
- `pass` → empty statement

Lab Exercise:-

1. Practical Example 1: Print fruits except "banana" using continue.

Program

```
list = ['apple', 'banana', 'mango']

for fruit in list:
    if fruit == "banana":
        continue
    print(fruit)
```

Output

```
apple

mango
```

Practical Example 2: Stop the loop once "banana" is found using break.

Program

```
list = ['apple', 'banana', 'mango']

for fruit in list:
    if fruit == "banana":
        break
    print(fruit)
```

Output

```
apple
```

9. String Manipulation

Theory:-

1. What is string manipulation?

String manipulation means accessing and modifying strings.

Common operations:

- Concatenation
- Repetition
- String methods (`upper()`, `lower()`)

2. What is string slicing?

String slicing extracts part of a string.

Lab Exercise:-

1. Write a program to demonstrate string slicing.

Program

2. Write a program using string methods.

Program

```
st = "sun rises in East"

print(len(st))
print(st.lower())
print(st.casefold())
print(st.upper())
print(st.title())
print(st.capitalize())
```

Output

```
17
sun rises in east
sun rises in east
SUN RISES IN EAST
Sun Rises In East
```

10. Advanced Python (map(), reduce(), filter(), Closures and Decorators)

Theory:-

1. What is functional programming in Python?

Functional programming uses functions to process data efficiently.

2. What are map(), reduce(), and filter()?

- `map()` → applies function to all elements
- `reduce()` → reduces list to single value
- `filter()` → filters elements based on condition

3. What are closures and decorators?

- **Closure** → function inside another function
- **Decorator** → modifies behavior of a function

Lab Exercise:-

1. Use map() to square list elements.

Program

2. Use reduce() to find product of numbers.

Program

3. Use filter() to filter even numbers.

Program